Progress Report: Protected Change-Tracking

Dennis E. Hamilton
Interoperability Architect
4401 44th Ave SW
Seattle, WA 98116, USA
+1-206-779-9430
dennis.hamilton@acm.org

ABSTRACT

Change-tracking of digital documents leads to document alteration in some manner. Protected Change-Tracking (PCT) is a response to the question, "How can one protect word-processing documents against alteration, verifiable by digital signature, and make signed amendments that are verified to be allowed modifications of the original document?" A genealogical model is identified that preserves the digital signature of the ancestor and each change-tracked descendant. PCT achieves a subset of genealogical cases having practical simplicity. Applicability to documents in ODF format is sketched.

Categories and Subject Descriptors

H.4.1 [**Information Systems Applications**]: Office Automation – *word processing;* H.1.2 [**Models and Principles**] User/Machine Systems – *human factors*, I.7.1 [**Document and Text Processing**]: Document and Text Editing – *version control, document management*

General Terms

Algorithms, Design, Management, Standardization, Verification

Keywords

OpenDocument, change tracking, document formats, digital signatures, change-protected documents, WYSIWYG.

1. INTRODUCTION

Protection of document files by digital signature and having change-tracked derivatives that preserve the protection is characterized with a genealogical protection model (section 2).

Being able to authenticate document files derived from protected ones and themselves protected has strong requirements for the verifiable reproducibility of one being derived from the other. PCT Protected Change Tracking isolates useful cases for which authentication is straightforward (section 3).

PCT can build on OpenDocument format. However, PCT representation requires signed OpenDocument Packages in a form different than existing ODF 1.2 document types (section 4).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyright for components of this work owned by others than ACM must be honored. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

DChanges '14, September 16, 2014, Fort Collins, CO, USA
Copyright is held by the owner/author(s). Publication rights licensed to ACM.
ACM 978-1-4503-2964-4/14/09…$15.00
http://dx.doi.org/10.1145/2723147.2723156

2. GENEALOGICAL PROTECTION

The genealogical protection model captures successions of digitally-signed and verifiable modifications to an initial protected document file.

2.1 Notation: Patterns of Manifestation

Following [3], $\mathbf{M}_x d$ symbolizes manifestation of a document via a particular configuration, x, of software, hardware, and user interface. Manifestations are what users perceive as bearing their digital signatures. Yet manifestations are intangible.

In contrast, d signifies the computer file that would be emitted (i.e., via a save operation) as a persistent artifact in a suitable format for digital interchange. If there is a digital signature, it is d that the signature is actually derived from and to which that signature is affixed

Emitting the document file is signified $\mathbf{M}_x d \rightarrow d$. Remanifestation in some (other) configuration is signified $\mathbf{M}_x d \rightarrow d \rightarrow \mathbf{M}_y d'$, abbreviated $\mathbf{M}_x d \rightrightarrows \mathbf{M}_y d'$.[1] When there is sufficient fidelity between $\mathbf{M}_x d$ and $\mathbf{M}_y d'$ they are said to be *essentially equivalent*, symbolized $\mathbf{M}_x d \approx \mathbf{M}_y d'$.

Essential equivalence does not require that presentations to users be strictly identical. Many incidental variations are tolerated.

Using "conformance" to apply to data structures and (relative) "compliance" to apply to processes, establishment of interoperability is accomplished by identifying an *envelope* such that for *envelope-compliant* x and y, if the *envelope-conformant* document files d and d' are such that $\mathbf{M}_x d \approx \mathbf{M}_y d'$ is satisfied, d, d' are said to be *manifestation-preserving* relative to the envelope, signified $d \cong d'$.

2.2 Document Genealogy

In what follows, genealogy is grounded on persistent artifacts: data representations in file formats. Envelope-conformance is expected (however actually accomplished). Within that context, the manifestation patterns are abbreviated, allowing the basic formulation to stand out.

Let d designate a complete document file, c be a digital record of one or more specific changes made together, with any additional supporting material, and g be a digitally-signed generation file

[1] For simplicity, assume the same file format absent any specific indication.

that consists of changes and a signed, unaltered previous generation.

A document genealogy, g_n, is a document file comprising a pair of components such that,

$$g_n = \langle c_n, g_{n-1} \rangle,\ n>0, \text{ and where, for } n \geq i>0,$$

[G1]: g_0 is the genealogy's base document file, d_0, the ancestor of the lineal descendant generations.

[G2]: g_i is the i-th descendant and the document file, d_i, is determined by it based on c_i and d_{i-1}.

[G3]: $\mathbf{M}d_j$, $n \geq j \geq 0$, is construed as the re-manifestation of d_j (i.e, some $d_j \rightarrow \mathbf{M}_y d_j'$ manifestation under envelope-compliance, not to be further belabored) as observed for inspection and understanding.

[G4]: c_i are constructed such that it is confirmable that $\mathbf{M}d_i$ is a descendant of $\mathbf{M}d_{i-1}$ in accordance with the details of c_i and g_{i-1} ([G2], above).

[G5]: g_{i-1}, $i>0$, is digitally signed; c_i may but need not be separately signed; and g_i is digitally signed.

[G6]: The signing of g_0 constitutes attestation that g_0 is protected and that $\mathbf{M}d_0$ descendants are limited to some prescribed and verifiable modifications.[2]

[G7]: The signing of g_i constitutes attestation that $\mathbf{M}d_i$ is intended and that $\mathbf{M}d_i$ descends from $\mathbf{M}d_{i-1}$ in accordance with c_i,g_{i-1} conditions ([G4], [G6], above).

[G8]: Authentication of g_i involves verification of the g_i signature, authentication of g_{i-1}, and confirmation that $\mathbf{M}d_i$ descends from $\mathbf{M}d_{i-1}$ in accordance with c_i,g_{i-1} conditions ([G4], above).

[G9]: $\mathbf{M}g_i$ is defined to be $\mathbf{M}d_i$.

2.3 Practical Constraints for Interchange

Envelope-conformance sets a floor on the envelope-compliance eligibility of processors as well as the kinds of changes and document features that are protectable.

Genealogical protection requires empirical confirmation of manifest equivalence, to within the envelope provisions, in the event of disputes. Absent such means, there is the prospect that the signer of g_{n-1} can refute that g_n is a proper descendant and the signer of g_n can refute that a claimed $\mathbf{M}_y g_n$ is equivalent to the $\mathbf{M}_x g_n$ they perceived themselves to be signing. To achieve some evidentiary standard, there must also be constraints on the genealogical artifacts and their processors such that inessential differences do not confound demonstration of essential equivalence.

3. PCT APPROACH

Protected Change Tracking (PCT) is a constrained application of the genealogical model. PCT applies to certain types of turnaround documents, such as word-processing-defined forms and spreadsheets used for data collection and suitable contract/agreement forms).

It is possible with existing spreadsheets and word-processing documents to specify protection against modification of all but allowed parts of documents. Unfortunately, the current password-based protection mechanisms used with office-productivity software are trivial to subvert and can be dangerous to use.[3] PCT employs signature-based protection, so that tampering voids the signature.

3.1 Limiting Conditions

The technical limitations arise from the following conditions.

1. The only complete document file in the genealogy is d_0, the ancestral document file. d_0 has unique identifiers affixed to those document-format components for which entries are allowed and to those whose values depend on the enterable ones. Initial values are usually present.

2. PCT protection begins with g_1. The c_1 item lists the initial value for each of the identified variable components of d_0 and establishes any conditions on subsequent generations and on capabilities essential to effective manifestations that account for all changes.

3. g_1 provides an identity stage for confirmation in interchange:

$$g_1 = [c_1, [d_0]]$$

with each pair of brackets signifying digital signing of everything within.

4. For subsequent generations, c_i enumerates only the changes made to identified components whether by entry or as a consequence (e.g., in a computed result).

$$g_i = [c_i, [c_{i-1}, [\ \ldots\ [c_1, [d_0]]\ \ldots\ ]]],\ i > 1$$

5. There shall be no overlaps or collisions among the identified fields. Identified fields cannot be removed or added. Acceptance or rejection of changes is a form of (provenance-preserving) new change.

6. The effective determination of d_i forms can be accomplished completely internal to the software as necessary to exhibit $\mathbf{M}d_n$ and provide authentication.

3.2 Authentication

The PCT procedure for authentication of g_n and manifestation of $\mathbf{M}d_n$ is as follows.

[A1]: Verify the generational signatures down through g_0.

[A2]: Starting with d_0, for each enterable field introduce the value for that field from the youngest (highest-index) c_i that identifies a value for that field. Use the already-present c_1 value if there is no younger one.

[A3]: Perform any necessary recalculation/update process to have the input-dependent results determined. This is the candidate d_n.[4]

---

[2] There can be additional signatures on the same generation file, reflecting attestations to additional conditions, possibly by additional parties, as in a negotiation approval and notarization process (Figure 5).

[3] https://tools.oasis-open.org/version-control/svn/oic/Advisories/00009-ProtectionKeySafety/trunk/description.html

[4] Arriving at d_n in this streamlined manner is based on assumption that this provides the $\mathbf{M}d_n$ that satisfies the creator of g_n. Otherwise the full recursive authentication procedure of [G8] is required.

[A4]: Confirm that d_n has no changes other than in the identified components.

[A5]: Verify that each identified component now has the same value as the youngest c_i for that component.

Manifest equivalence is almost assured by the confinement of changes, their effects, and the manner in which d_n is obtained from d_0, so long as $\mathbf{M}_x d_n \approx \mathbf{M}_y d_n$ between originator and receiver. Two additional measures protect against deviations among software implementations (and falsified c_i).

[A6]: For input entries, retain the value as entered and, if different, the presentation form by which the entry is presented for view. This is important for numeric and other fields, such as calendar dates and currency-value formats where what is entered/stored and what is presented differ. Discrepancies between $\mathbf{M}d_n$ and the youngest entry's presentation form must be investigated.

[A7]: Similarly for derived-value components, retain the value as derived internally at time of generation creation and also the presentation form by which the derivative value is manifest (e.g., with any text formatting carried in the document file). Discrepancies between the recalculated/updated $\mathbf{M}d_n$ presentation and the youngest-result presentation form must be investigated.

4. SUPPORTING PCT IN ODF

The ODF 1.2 document-file format already supports digital signature of OpenDocument document files implemented as OpenDocument Packages [5]. ODF packages consist of Zip files that contain the multiple parts satisfying the ODF specification and from which the document is re-manifest [6]. Digital signature of ODF 1.2 packages is by incorporation of a single ODF document signature file based on XML Digital Signature [1][2]). `META-INF/documentsignatures.xml` files provide the signature of all other files within signed ODF packages (Figure 4). The document-signature file itself has internally-signed components such that critical parts cannot be manipulated without invalidating the signature.

The existing package and digital-signature provisions of ODF 1.2 afford simplified adaptation for PCT.

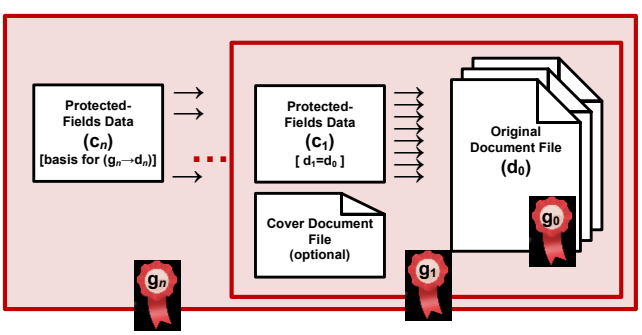

Figure 1. Simplified ODF PCT Structure. Each c_i provides only changes for the corresponding generation, covering over any earlier entry for the same protected-field.

4.1 Preparing the First Generation

The base generation d_0 can be a conventional ODF document file which is locked in a manner that leaves only certain components available for alteration, such as entry of a form field. Affixing identifiers to those components and any derivative components is part of PCT preparation (Figure 2), resulting in an OpenDocument Extended Document ([4], section 2.2.2). The d_0 ODF document file is a digitally-signed package with OpenDocument document signature ([5], section 3.16).

```
<table:table-cell
        pct:cell-label="Sheet1.C1"
        pct:cell-type="pct:cell-derived"
        table:formula="of:=SUM([.A$1:.A1])" office:value-type="float"
        office:value="0.714173431973904">
    <text:p>0.714173432</text:p>
</table:table-cell>

<table:table-cell
        pct:cell-label="Sheet1.A2"
        pct:cell-type="pct:cell-input"
        table:style-name="ce2" office:value-type="float"
        office:value="3.14159265358979">
    <text:p>3.1415926536</text:p>
</table:table-cell>

<table:table-cell
        pct:cell-label="Sheet1.C2"
        pct:cell-type="pct:cell-derived"
        table:formula="of:=SUM([.A$1:.A2])" office:value-type="float"
        office:value="3.85576608556369">
    <text:p>3.8557660856</text:p>
</table:table-cell>
```

Figure 2. c_1: $d_{0/1}$ Protected-Change List. The table-cell elements for protected cells are exact replicas of the corresponding elements, with PCT markers, in the base document, d_0.

Next, d_0 is incorporated into another digitally-signed ODF 1.2 package along with a change-stack structure having c_1 as its only entry. This specialized g'_1 file is also an ODF 1.2 package. It is not a standard ODF 1.2 document file. The stream-lined form is

$$g'_1 = [s_1\text{: } d', \langle c_1 \rangle, [d_0]]$$

where s_1 signifies the digital signature file, d' is an ODF cover document that non-PCT processors recognize the generation as, manifesting as a PCT-is-required warning (Figure 3), and $[d_0]$ is the already-signed d_0 embedded in g'_1 as a single part. The signature also covers a package packing-list file, `META-INF/manifest.xml`, that identifies all of the parts (except the signature file) and is required in every package.

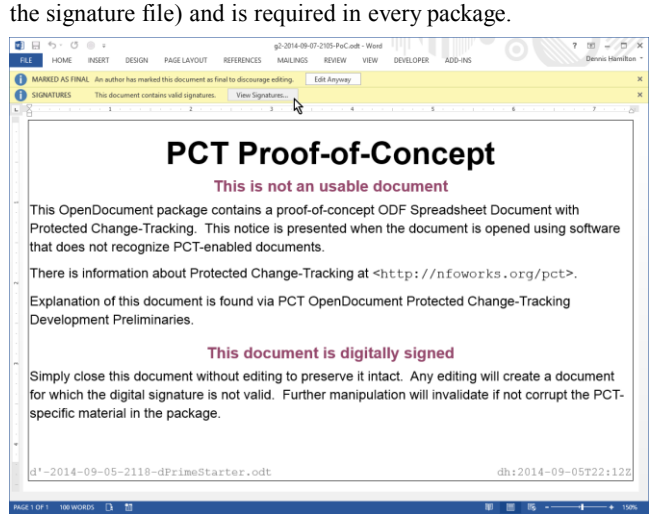

Figure 3. Representative Cover Document

4.2 Subsequent Generations

In preparation of g'_{i+1} from g'_i, s_i and the signed manifest file, m_i are inserted on the change stack followed by c_{i+1}. The modified

ODF 1.2 package is digitally signed using an ODF Digital Signature (Figure 4).

$$g'_{i+1} = [s_{i+1}: d', \langle c_{i+1}, m_i, s_i, \ldots, c_1\rangle, [d_0]],\ i > 0.$$

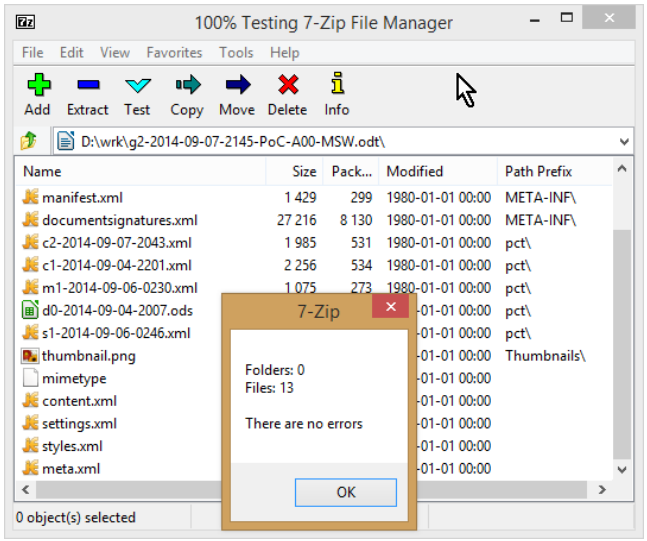

Figure 4. ODF PCT Spreadsheet Zip Structure. The PCT stack consists of pct\-prefixed parts. The top-level signature applies to the cover document and all of the subordinate parts. Current ODF signature implementations sign all content without regard to what the cover document requires.

Verifying ancestry of g'_{i+1} is by effectively unwinding the change-stack one level, restoring manifest and signature files to their locations, and verifying the ancestry of the resulting parent generation, one older generation at a time (Figure 1).

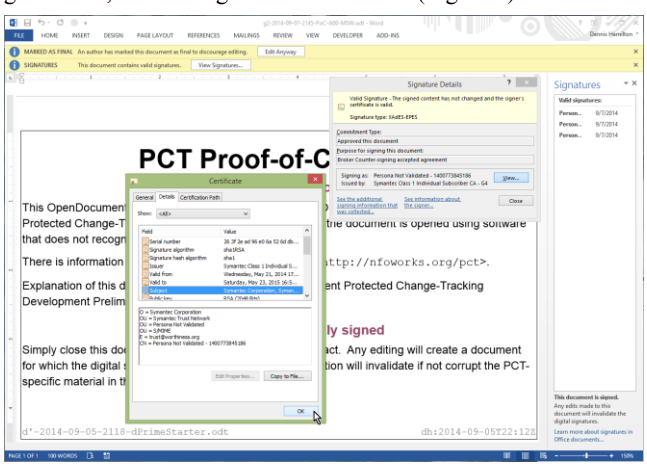

Figure 5. Demonstration of Multiple Signatures on Current Generation. Using existing ODF provisions, the current generation can be signed and counter-signed as part of provenance affirmation and notarization or for any other purpose.

5. CONCLUSION

PCT and especially the ODF 1.2 specialization appear feasible in terms of the ability to rely on mechanisms of existing implementations (Figure 5).

Taking ODF 1.2 as a thought experiment, the packaging of the g'_i, is the easiest part. Practicability depends on the ease with which existing implementations can be adjusted to produce $[d_0]$ and successive generation files. Verification of generations requires new controls over editing and capture of changes observed as the effects of the allowed entries.

It is uncertain to what degree existing implementations and their implementers are amenable to introduction of such capabilities. The level of difficulty for implementation by modification of existing software is unknown. A proof-of-concept investigation is required to gauge the difficulty of identifying and isolating points of change for dependable introduction of PCT support in released software. Proof-of-concept development and investigation of implementation feasibility will be conducted on a public web site.[5]

In terms of standardized use, one can envision specification of community-ratified profiles that supplement ODF 1.2 (for example) and establish particular classes of PCT envelope-compliance along with conditions for $\mathbf{M}_x d \approx \mathbf{M}_y d'$ conformance within each class. This may be the ultimate barrier. The requirement for conformance testing of manifest equivalence may be just as daunting as for unprotected forms of change-tracking (e.g., the repaired change-tracking for ODF, RCT [3]).

6. ACKNOWLEDGMENTS

Thanks to the anonymous reviewers for their comments and effort and to the participants in the DChanges 2014 Workshop for their spirited and inspiring discussions. Thanks to colleague William L. Anderson for his enthusiastic attention and careful eye.

7. REFERENCES

[1] Donald Eastlake, Joseph Reagle, David Solo, Frederick Hirsch, and Thomas Roessler (editors). XML Signature Syntax and Processing (Second Edition), W3C Recommendation 10 June 2008. http://www.w3.org/TR/2008/REC-xmldsig-core-20080610/

[2] ETSI. XML Advanced Electronic Signatures (XAdES), ETSI TS 101 903 v1.4.1 2009-06-15. http://webapp.etsi.org/workprogram/Report_WorkItem.asp?WKI_ID=28064

[3] Dennis E. Hamilton. Tracked Changes: Navigating the Document-Format Anti-Pattern. *DChanges '14*, September 16, 2014, Fort Collins, Colorado, USA http://dx.doi.org/10.1145/2723147.2723153

[4] OASIS. *Open Document Format for Office Applications (OpenDocument) Version 1.2 Part 1: OpenDocument Schema*. 29 September 2011 OASIS Standard. Available at http://docs.oasis-open.org/office/v1.2/os/

[5] OASIS. *Open Document Format for Office Applications (OpenDocument) Version 1.2 Part 3: Packages*. 29 September 2011 OASIS Standard. Available at http://docs.oasis-open.org/office/v1.2/os/

[6] PKWARE. *APPNOTE.TXT - .ZIP File Format Specification Version 6.2.0*, 2004-04-26. Available at http://www.pkware.com/support/application-note-archives

[5] http://nfoworks.org/pct